LITTLE STREAM DETECTOR

LSD 2.0

Detailed Technical Documentation

Document type	Architecture and implementation reference
Release	2.0 Final Release
Reference build	LSD-2.0-final-release.ps1
Build SHA-256	7feeb7de52dad5bc78fa5ca53c88acc5ea3e683689582ce780dfdfe2b3a8e9d2
Documentation date	20 August 2026

Scope This document describes the native container, codec, canonical-sample, analysis, validation, reporting, threading, and safety architecture implemented in the referenced final release. It is intended for maintainers, reviewers, and future contributors.

No external multimedia tools or temporary files are required at runtime.

Contents

1. Purpose and System Scope

Little Stream Detector 2.0 is a native Windows PowerShell application with an embedded C# analysis engine. The application inspects media containers, builds a container-neutral sample index, performs codec-aware bitstream analysis, and presents both human-readable and structured results through a responsive graphical interface.

The design intentionally avoids external multimedia processes at runtime. Container parsing, sample indexing, codec-private parsing, frame classification, quantizer extraction, audio validation, bitrate calculation, and report generation are performed in memory by the script and embedded C# types.

Design principle A container determines where media samples are stored. A payload format determines how codec units are packaged inside each sample. A codec analyzer determines what those units mean. The canonical model separates these responsibilities.

1.1 Primary objectives

- Provide deterministic native analysis without shelling out to FFmpeg, MediaInfo, or other multimedia tools.
- Keep container-specific addressing separate from codec-specific interpretation.
- Validate every sample against file boundaries before reading payload data.
- Report unavailable information as N/A rather than guessing.
- Support asynchronous preparation and analysis without blocking the GUI.
- Retain compatibility with Windows PowerShell 5.1 and in-memory EXE hosting.

1.2 Supported families

Area	Supported families
Containers	Matroska, WebM, AVI 1.0, OpenDML AVI 2.0, MP4, M4V, MOV
Video	H.264/AVC, H.265/HEVC, AV1, MPEG-4 Part 2/XviD
Audio	AAC-LC, HE-AAC, HE-AACv2, MP3, PCM integer/float, AC-3, E-AC-3
Outputs	Summary, Streams, JSON, Log, bitrate profile, quantizer histogram

2. Architectural Overview

LSD is physically distributed as a single PowerShell file, but logically behaves as a modular monolith. The PowerShell layer owns application state, presentation, orchestration, and background-job lifecycle. Embedded C# types own binary parsing, canonical models, bitstream traversal, validation, and numerical accumulation.

```
User / GUI
   |
   v
Metadata and job coordinator
   |
```

```
  +--> Matroska probe and adapter
  +--> MP4/MOV probe and adapter
  +--> AVI/OpenDML probe and adapter
            |
            v
       Canonical media model
            |
    +-------+--------+
    |                |
    v                v
Video analyzer      Audio analyzer
    |                |
    +-------+--------+
            v
    Diagnostics, reports, graphs
```

2.1 Logical layers

Layer	Responsibility	Representative components
Presentation	Window, controls, reports, graphs, drag-and-drop, copy actions	PowerShell WinForms
Coordination	Metadata preparation, state transitions, cancellation, progress	Start-Meta, Complete-NativeMetadata, Start-Scan
Container parsing	Detect structure, read tracks, resolve sample locations	NativeMediaProbe, NativeMp4Probe, NativeAviProbe
Canonical adaptation	Normalize tracks and samples into common models	MatroskaToLsdAdapter, Mp4ToLsdAdapter, AviToLsdAdapter
Codec analysis	Parse codec configuration and frame-level syntax	AVC, HEVC, AV1, MPEG-4 Visual, AAC, MPEG Audio, Dolby, PCM
Diagnostics	Count parsed/rejected units, retain first failure, test completeness	Analysis result and diagnostic objects
Reporting	Compose readable and structured output	Build-NativeMetadata and GUI views

2.2 Why a canonical pipeline is important

The same H.264 slice parser is used whether the original sample came from a Matroska block, an MP4 sample table, or an AVI/OpenDML index. Container adapters normalize offsets, lengths, timestamps, keyframe flags, and payload packaging. This prevents codec analyzers from duplicating container logic and allows new container routes to reuse existing analyzers.

3. Runtime and Processing Lifecycle

3.1 Startup

- The embedded C# source is compiled through Add-Type.
- Native self-tests validate selected arithmetic and bit-reader behavior.
- The console window is hidden for GUI operation.
- Invariant English culture is applied to keep decimal and report formatting deterministic.
- Global application state is initialized for metadata, prepared models, jobs, diagnostics, and graphs.

3.2 File processing phases

Phase	Action	Output
1. Selection	User selects or drops a file	Absolute source path
2. Native detection	Inspect header bytes and identify container family	NativeMediaInfo
3. Metadata preparation	Parse tracks, configuration, metadata, and sample tables/indexes	Container probe result
4. Canonical adaptation	Create common tracks and canonical samples	LsdMediaModel
5. Codec analysis	Read bounded samples and parse frame/audio syntax	Frame and audio diagnostics
6. Finalization	Build statistics, graphs, and readable reports	GUI views and copyable text

3.3 Background jobs

MP4/MOV and AVI preparation run in dedicated background threads because sample-table and index construction can require many entries. Codec analysis also runs in a background worker. The GUI timer polls job progress, updates the progress bar and status text, and finalizes results after completion. Cancellation flags are checked during long-running loops, and disposal joins worker threads for a bounded interval.

Threading boundary The GUI does not parse media payloads directly. Background jobs produce immutable or completed result objects before the PowerShell layer publishes them to report and graph controls.

4. Canonical Media Model

4.1 LsdMediaModel

The top-level canonical object represents the source file independently of its original container.

Field	Meaning
SourcePath	Input file path
ContainerType	Normalized container family
ContainerDetail	Specific container/index route
FileLength	Source length in bytes
DurationSeconds	Best available duration
TimecodeScale	Nanoseconds represented by one canonical timestamp tick

Tracks	Canonical video, audio, subtitle, and other tracks

4.2 LsdTrackModel

Field	Meaning
TrackId	Container track or stream identifier
TrackType	video, audio, subtitle, or other
CodecId	Container-native codec identifier
CodecFamily	Normalized analyzer family
CodecPrivate	Codec initialization record when available
PayloadFormat	Physical packaging of each sample
NalLengthSize	Length-prefix width for AVC/HEVC
PCM fields	Bit depth, block alignment, float flag, endianness, sample rate
Samples	Ordered canonical sample list

4.3 LsdSampleModel

Field	Meaning
FileOffset	Absolute payload offset in the source file
Length	Payload size in bytes
DecodeTimestamp	Decode-order timestamp
PresentationTimestamp	Presentation-order timestamp
Duration	Sample duration when known
IsKeyFrame	Container keyframe or sync indication
IsInvisible / IsDiscardable	Matroska block flags where available
PayloadFormat	Packaging used by the codec reader

4.4 Payload formats

Payload format	Purpose
LengthPrefixedNal	AVC/HEVC NAL units preceded by configured lengths

AnnexBNal	Start-code-delimited AVC NAL units in AVI
Av1Obu	AV1 Open Bitstream Units carried by a sample
Mpeg4Visual	MPEG-4 Visual start codes and VOP payload
MpegAudio	MPEG Audio frame payload
RawCodecFrame	Raw AAC, Dolby, PCM, or other sample payload

5. Container Modules

5.1 Matroska and WebM

The Matroska reader parses EBML elements, Segment metadata, Tracks, Cluster timing, Block and SimpleBlock structures, and supported lacing modes. TrackEntry data supplies codec IDs, language, dimensions, audio rate/channels, codec-private data, and PCM bit depth. MuxingApp and WritingApp are reported separately.

- Canonical video and audio samples are created from Block/SimpleBlock payload ranges.
- No lacing, fixed lacing, Xiph lacing, and EBML lacing are handled.
- Block boundaries are validated before payload traversal.
- Matroska PCM IDs A_PCM/INT/LIT, A_PCM/INT/BIG, and A_PCM/FLOAT/IEEE are normalized into the PCM family.

5.2 MP4, M4V, and MOV

The ISO Base Media reader supports non-fragmented files with a moov-based sample table. It traverses track hierarchy and derives sample offsets, sizes, decode times, presentation offsets, and sync flags.

Box/table	Use
stsd	Sample entry and codec configuration
stsz	Individual or common sample sizes
stsc	Samples-per-chunk mapping
stco / co64	32-bit or 64-bit chunk locations
stts	Decode timestamps
ctts	Composition offsets
stss	Sync sample set
udta / meta / ilst	Writing application metadata

QuickTime SoundSampleEntry versions 0, 1, and 2 are accepted. Supported audio entries include AAC/MP3/Dolby and PCM forms such as sowt, twos, in24, in32, fl32, fl64, and lpcm. Fragmented MP4 structures are intentionally outside the 2.0 scope.

5.3 AVI 1.0 and OpenDML AVI 2.0

The AVI module reads RIFF AVI headers, stream headers and formats, LIST/INFO metadata, movi areas, legacy idx1 entries, OpenDML AVIX segments, and standard ix## indexes. It chooses OpenDML indexes when valid and falls back to idx1 otherwise.

```
OpenDML preferred path
RIFF AVI + RIFF AVIX
   -> ix00 / ix01 / ix##
   -> 64-bit base offset + relative entry offset
   -> bounded chunk resolution
   -> deduplicated canonical samples

Legacy fallback
RIFF AVI -> idx1 -> canonical samples
```

- Multiple top-level RIFF segments are inspected.
- AVIX segment count is recorded for diagnostics.
- OpenDML base offsets are represented as 64-bit unsigned values and checked before conversion.
- Zero-length entries are skipped before counters are advanced.
- A HashSet of payload offsets prevents duplicate accounting.
- Alternative legacy offset conventions are accepted only when the chunk FourCC and declared size validate.
- AVI AVC streams use Annex B payloads and discover SPS/PPS from early video samples.

5.4 Container metadata

Container	Metadata handled
Matroska/WebM	MuxingApp and WritingApp
AVI	LIST/INFO/ISFT software string
MP4/MOV	Copyright too and QuickTime swr metadata entries

6. Video Analysis Modules

6.1 H.264 / AVC

- avcC parsing for profile, level, NAL length width, SPS, and PPS.
- Annex B scanner for AVI start-code-delimited NAL units.
- SPS/PPS discovery from AVI video payloads.
- I, P, and B slice classification.
- Native SliceQPY extraction from slice headers.
- CABAC/CAVLC-aware parsing context and PPS-derived defaults.
- VUI-derived pixel aspect and color metadata when explicitly signaled.
- Packet-to-frame consistency and per-type QP statistics.

6.2 H.265 / HEVC

- hvcC parsing for profile, tier, level, temporal layers, chroma, bit depth, and NAL length width.
- VPS/SPS/PPS context probing.
- IRAP, IDR, CRA, BLA, TRAIL, RADL, and RASL classification.
- First-slice I/P/B classification and native SliceQPY analysis.

- PPS context for weighted prediction, references, tiles, WPP, deblocking, and list modification.
- Optional VUI-derived aspect and color data.

6.3 AV1

- av1C configuration validation and OBU inventory.
- Sequence Header interpretation, profile/level/tier, dimensions, chroma, bit depth, color, order hints, and tool flags.
- Frame Header and Frame OBU traversal.
- KEY_FRAME, INTER_FRAME, INTRA_ONLY_FRAME, and SWITCH_FRAME accounting.
- Hidden frames, showable frames, and show_existing_frame events.
- Frame-state and geometry accounting.
- Single-tile frame-level Base Q Index statistics and histogram.

6.4 MPEG-4 Part 2 / XviD

- MPEG-4 Visual Object Layer and VOP start-code traversal.
- I, P, B, and non-coded VOP handling.
- Packed-sample detection.
- VOP quantizer extraction and histogram.
- Packet/frame consistency for AVI and Matroska routes.

7. Audio Analysis Modules

7.1 AAC

AAC configuration is derived from AudioSpecificConfig. The parser identifies core object type, sample rate, channel configuration, SBR, and Parametric Stereo. User-facing naming distinguishes AAC-LC, AAC-HE, and AAC-HEv2. Container and codec values remain separate so core and output rates/channels can be reported correctly.

7.2 MPEG Audio / MP3

- MPEG version and layer detection from frame headers.
- Bitrate index, sample-rate index, padding, channel mode, and frame-size validation.
- Decoded samples per frame and exact decoded-duration accounting.
- Exact bitrate from validated frame bytes divided by decoded duration.
- Minimum and maximum frame bitrate reporting.

7.3 AC-3 and E-AC-3

- Syncword scanning and codec distinction through bitstream ID.
- Sample rate, frame size, block count, channel mode, LFE, and layout extraction.
- Decoded PCM sample accounting.
- Exact bitrate from validated Dolby frame bytes and decoded duration.
- dac3 and dec3 codec-private parsing where carried by MP4/MOV.

7.4 PCM

PCM analysis validates structure rather than compressed frame syntax. The canonical analyzer verifies that every sample length is divisible by block alignment, accumulates payload bytes, calculates decoded duration, and compares expected and measured bitrate.

```
blockAlign = channels * ceil(bitDepth / 8)
decodedDuration = payloadBytes / blockAlign / sampleRate
bitrate = sampleRate * bitDepth * channels
```

Container	Supported PCM identification
AVI	WAVE_FORMAT_PCM, WAVE_FORMAT_IEEE_FLOAT, supported WAVE_FORMAT_EXTENSIBLE parameters
Matroska	A_PCM/INT/LIT, A_PCM/INT/BIG, A_PCM/FLOAT/IEEE
MOV/ISO Media	sowt, twos, in24, in32, fl32, fl64, lpcm

8. Quantizer and Bitrate Analysis

8.1 Bitrate profile

For each canonical video sample, LSD records byte size and maps the sample presentation time into a fixed set of bitrate bins. The GUI renders these bins as the upper bitrate profile. The source is container-neutral: Matroska blocks, MP4 samples, AVI idx1 entries, and OpenDML ix## entries all feed the same accumulator.

8.2 Quantizer sources

Codec	Native value	Histogram
AVC	SliceQPY	Frame-level SliceQPY distribution
HEVC	SliceQPY	Frame-level SliceQPY distribution
AV1	Base Q Index	Frame-level Base Q Index distribution
MPEG-4 Part 2	VOP quantizer	VOP quantizer distribution

8.3 Completeness rules

- A value is considered complete only when the required native parser succeeds for every relevant frame.
- Rejected syntax units increment explicit counters and preserve the first error message.
- No estimated quantizer values are substituted.
- A GOP interval is reported only when at least two keyframes exist; otherwise the output is N/A.

9. GUI, Jobs, and Reporting

9.1 Application state

PowerShell script-scoped variables track the selected file, native metadata, prepared media model, active jobs, scan phase, packet/frame results, bitrate bins, quantizer counts, and codec-specific diagnostics. State is reset for each new file before metadata preparation starts.

9.2 Views

View	Content
Summary	Readable file, track, canonical, codec, audio, diagnostic, and engine sections
Streams	Structured per-stream data derived from native tracks
JSON	Serializable metadata and stream representation
Log	Lifecycle messages and validation status
Graphs	Video bitrate profile and codec-specific quantizer distribution

9.3 Report design rules

- Container-native identifiers are preserved in Native Container Tracks.
- Main sections use readable codec names such as PCM signed integer and MPEG-4 Part 2 / XviD.
- N/A is used when data is absent, unsignaled, or not applicable.
- Units are emitted only when a numeric value exists.
- OpenDML and legacy AVI index routes are identified explicitly.
- The report distinguishes payload bytes from codec-private configuration bytes.

10. Validation and Defensive Parsing

10.1 Boundary checks

- Every canonical sample must have a non-negative offset and positive length.
- offset + length must remain inside the source file.
- Container chunk and box sizes are checked for overflow and parent-bound violations.
- OpenDML 64-bit base offsets are checked before signed conversion.
- Index entry counts must fit inside the index chunk size.
- NAL, OBU, MPEG Audio, Dolby, and PCM readers reject truncated payloads.

10.2 AVI-specific protections

- Zero-length video and audio index entries are ignored.
- OpenDML and legacy entries are not merged into duplicate samples.
- Resolved chunks must match the expected stream FourCC and minimum declared size.
- OpenDML is preferred only when at least one usable standard-index sample is obtained.
- Legacy idx1 remains a controlled fallback.

10.3 Error model

Non-critical optional metadata parsers may fail without invalidating the core analysis path. Critical sample, index, and payload violations produce a controlled error or an INCOMPLETE result. Diagnostic objects expose rejected counts and the first failure to avoid silent data loss.

Safety rationale The code intentionally retains several narrow fallback and exception boundaries around optional VUI data and historical AVI offset conventions. Removing these blocks would reduce

compatibility and is not considered safe cleanup.

10.4 Runtime isolation

- No external multimedia executable is launched.
- No temporary media files are created.
- No PATH lookup is required.
- The source file is opened read-only with shared read access.
- The application can be hosted from memory by an EXE wrapper.

11. Supported Combinations

11.1 Container and video matrix

Video	Matroska/WebM	MP4/M4V/MOV	AVI
H.264 / AVC	Yes	Yes	Yes, Annex B
H.265 / HEVC	Yes	Yes	No
AV1	Yes	Yes	No
MPEG-4 Part 2 / XviD	Yes	Not enabled	Yes

11.2 Container and audio matrix

Audio	Matroska/WebM	MP4/M4V/MOV	AVI
AAC-LC / HE-AAC / HE-AACv2	Yes	Yes	No
MP3	Yes	Yes	Yes
PCM integer / float	Yes	Supported entries	Yes
AC-3	Yes	Yes	Yes
E-AC-3	Yes	Yes	No

11.3 PCM variants

Variant	Status
Signed integer little-endian	Supported
Signed integer big-endian	Supported where signaled
IEEE floating-point	Supported

8/16/24/32-bit integer	Supported when container parameters are complete
32/64-bit float	Supported for corresponding container entries

12. Known Limitations

- Fragmented MP4 structures using moof, traf, and trun are not supported in version 2.0.
- AVI H.264 support targets Annex B payloads. Unusual length-prefixed AVC storage in AVI may not be accepted.
- AVI AAC and AVI E-AC-3 audio are not enabled.
- Container-only color metadata, including MP4 colr/nclx and Matroska Colour elements, is not used as a fallback when the codec bitstream does not signal color information.
- Certain HEVC PPS syntax involving scaling-list data remains intentionally incomplete and is reported as such.
- OpenDML field-index subtype layouts beyond the supported standard chunk index are not treated as canonical frame samples.
- Metadata absent from both container and codec bitstream is reported as N/A.

13. Verification and Regression Matrix

The final release was validated with focused reference assets that exercise both new and legacy paths. The table below records the expected high-level result for the closing test suite.

Reference scenario	Primary path verified	Result
OpenDML AVI + AVIX + XviD + PCM S16LE	ix00/ix01, AVIX, deduplication, legacy idx1 coexistence	PASS
AVI 1.0 + XviD + PCM S16LE	idx1 fallback, PCM WAVEFORMATEX	PASS
Matroska + AVC + PCM S24LE	Blocks/lacing route, A_PCM/INT/LIT, 24-bit alignment	PASS
MOV + AVC + PCM S16LE	ISO sample tables, sowt, per-sample accounting	PASS
MOV + AVC + PCM F32LE	ISO sample tables, fl32, floating-point PCM	PASS

13.1 Required invariants

- Canonical sample counts equal the corresponding validated index or sample-table counts.
- Packet/frame match is Yes for completed video analyses.
- Rejected audio samples are zero for conforming references.
- Expected and measured PCM bitrate match.
- PCM decoded duration matches source duration.
- Quantizer completeness is PASS when one native value is obtained per frame.
- Both bitrate and quantizer graphs complete after background processing.

13.2 Final release audit

Audit item	Outcome
Embedded C# lexical structure	PASS
PowerShell 5.1 compatibility constraints	PASS
Development console output	Removed
External process/path dependencies	None
Unsafe numeric plus text expressions	None detected
Malformed N/A plus unit output	None detected
Dead throughput benchmark route	Removed
Defensive parsing fallbacks	Retained

14. Maintenance Guidelines

14.1 Safe extension sequence

- Add or update container-native codec mapping.
- Normalize the codec family and payload format.
- Populate canonical tracks and samples without codec-specific interpretation.
- Add or reuse a payload reader.
- Invoke an existing codec analyzer or add a new isolated analyzer.
- Expose parsed/rejected counts and first failure.
- Add a report section only when the corresponding codec family is active.
- Create a small deterministic regression asset.
- Verify both the new path and at least one existing path in the same container.

14.2 Code that should remain stable

- Canonical model contracts.
- Sample boundary validation.
- AVI/OpenDML chunk resolution checks.
- MP4 parent-bound box validation.
- Matroska lacing size accounting.
- Codec parser completeness rules.
- Background job cancellation and disposal.
- N/A reporting policy.

14.3 Future candidates

- Fragmented MP4 support.
- Container-color fallback through MP4 colr/nclx and Matroska Colour.
- Optional AVI AAC/E-AC-3 mapping.
- Additional video codecs such as VP9 or MPEG-2 Video.

- Formal static registries for container and codec modules while retaining the single-file deployment model.

Appendix A. Key Types and Responsibilities

Type	Responsibility
NativeMediaInfo / NativeTrack	Container-native metadata and codec configuration state
LsdMediaModel / LsdTrackModel / LsdSampleModel	Container-neutral tracks and bounded samples
NativeMediaProbe	Initial detection and Matroska metadata parsing
NativeMp4Probe	ISO Media metadata and sample-table construction
NativeAviProbe	AVI 1.0/OpenDML parsing and canonical index construction
NativeMatroskaPacketScan	Historical name for the shared canonical video analysis job
CanonicalAudioAnalyzer	MP3, Dolby, and PCM sample validation and accounting
H264SliceHeaderParser	AVC slice classification and SliceQPY
HevcHvccContextProbe	HEVC parameter-set context
Av1CodecConfigurationProbe	AV1 configuration and sequence data
PowerShell coordinator	Jobs, state, reports, graphs, and GUI

Appendix B. Terminology

Term	Definition
Canonical sample	Container-neutral record describing an absolute payload range and timing.
Codec private	Initialization/configuration data stored outside ordinary media samples.
Payload format	Method used to package codec units inside a canonical sample.
Annex B	Start-code-delimited H.264/H.265 byte-stream representation.
OpenDML	AVI extensions supporting AVIX segments and scalable indexing.
VOP	MPEG-4 Part 2 Video Object Plane.
SliceQPY	Luma quantization parameter derived from AVC/HEVC slice

	syntax.
Base Q Index	AV1 frame-level base quantizer index.
Block alignment	Bytes required for one interleaved PCM sample frame across all channels.
Complete	All required units parsed without rejection for the relevant result.

Appendix C. Release Identification

This documentation corresponds to the following audited release artifact:

Property	Value
File name	LSD-2.0-final-release.ps1
Version	2.0 Final Release
SHA-256	7feeb7de52dad5bc78fa5ca53c88acc5ea3e683689582ce780dfdfe2b3a8e9d2
Runtime model	Single PowerShell file with embedded C#
Runtime dependencies	Windows PowerShell 5.1 and .NET Framework classes used by the application
External multimedia tools	None
Temporary media files	None

Release status The documented build is the closing LSD 2.0 release. Subsequent functional changes should be versioned as a later maintenance or feature release rather than silently replacing this artifact.